

Lendrail v2 – multi-lender point-of-sale finance orchestration

A clean-room **event-sourced monolith**: merchants embed a checkout widget, a waterfall router places each application with one of three simulated lenders behind a sealed adapter, and the platform runs the full lifecycle — offers, counter-offers, webhooks, settlement, disputes, and consumer loan servicing — with every state change folded from one append-only journal.

INVARIANT 01

Event-sourced

All state is a pure fold of one append-only JSONL journal. No SQL, no ORM.

INVARIANT 02

Sealed adapters

Lenders see DTOs only — never the store, clock, or HTTP. `assess()` is pure & replayable.

INVARIANT 03

Router records, never decides

The waterfall sequences legs and freezes a trace; it holds no outcome literal.

INVARIANT 04

Twin reconcilers

Settlement & servicing each pair a deterministic engine with an independent re-derivation.

INVARIANT 05

Virtual clock

Every timer runs on a controllable clock, re-armed from projections on boot.

01 System topology

Ten client surfaces reach the platform through three distinct connection styles. Ports are canonical, from packages/spec/conventions.json.

CLIENT SURFACES · APPS/

Server-side via SDK

merchant server → @lendrail/sdk

Store — Voltora

7410

Merchant storefront; embeds the checkout widget.
storeApi 7411

Store — Kestrel

7414

Second tenant of the same codebase
(LENDRAIL_STORE_TENANT)

Demo-cart — Larkspur

7415

Reference shop via @lendrail/integration-kit

Browser SPA → HTTP

fetch → platform :7401

Dashboard

7412

Merchant back-office: orders, refunds, 2FA admin

Lender Ops

7413

Lender console: referrals, disputes, settlement

Servicing

7440

Consumer loan-servicing PWA → /consumer/*

Static & embed

no live platform calls

Widget

embed

Checkout overlay; CAC + HAL-link driven, iframe steps

Portal

7432

Sandbox dev portal: keys, webhook tester

Explorer · Docs · Landing

7420 · 7430 · 7431

API reference, docs site, marketing reel

▼ SDK client-credentials auth · HAL+JSON · zod-validated · **rate-limited** ▼

@lendrail/platform · Fastify :7401

HTTP surface → command handlers → event store → in-memory projections

INGRESS

HTTP surface

http/server.ts — HAL, zod, error
taxonomy

DECIDE

Command handlers

platform.ts — validate, invoke
subsystems

APPEND

Event store

core/eventStore.ts — one serialized
writer

FOLD

Projections

domain/state.ts apply() — pure
reducer

JOURNAL

data/journal/events.jsonl — append-only truth · separate webhook-deliveries.jsonl · torn-tail repair · seq-gap refuses boot · encrypted snapshot cache · PII-at-rest codec

02 Platform kernel — subsystems

Command handlers dispatch into these services. Each writes journal events; queryable state is the fold. Two subsystems ship an *independent verifier* that re-derives the engine's claims from the raw journal.

■ Lenders

SEALED

- **adapter.ts** — DTO-only boundary:
`assess(input)→assessment`
- **policy.ts** — one deterministic engine; email-hash credit score, affordability, magic short-circuits
- **registry.ts** — priority-ordered lenders (boot config, not journaled)
- **router.ts** — waterfall: run leg's lender, probe fallbacks only on decline

■ Settlement

+ RECONCILER

- **engine.ts** — clock-triggered per-lender/tenant runs; negative carry-over; `SUCCESSFUL→SETTLED`; CSV + remittance
- **calendars.ts** — per-lender settlement windows
- **reconcile.ts** — re-folds the raw journal from scratch; diffs every column (two-derivation gate)

■ Webhooks

- **notifier / families** — map order & dispute status → wire vocabulary; dedupe per checkout
- **outbox.ts** — **HMAC-SHA256** over canonical JSON; virtual-clock retry ladder 10s → ... → 12h → FAILED
- secrets **derived, not stored**; rotation keeps history so rotated-away keys still verify

■ Servicing

+ RECONCILER

- **service.ts** —
`application.decided→loan.opened`; refunds re-split or close
- **schedule.ts** — instalment rows via finance-core
- **collection.ts** — Direct Debit timer wheel; arrears derived, late-fee ladder
- **hardship.ts** — holiday / restructure / reject

■ Disputes

- state machine INITIATED → REPRESENTMENT → PRE-ARB → ARBITRATION → CLOSED **WON/LOST**
- non-refundable dispute fee; reason-banded response deadline → auto-close LOST
- CLOSED-LOST emits reversal debiting the lender's next settlement

■ Identity & tenancy

- **auth/keys · tokens** — signing registry; JWT checkout/product/tenant claims (jose)
- **tenants** — multi-tenant registration, webhook config, secret rotation
- **admin · lenderops users** — RBAC; manual underwriting decisions

■ Money & offers

- **domain/state.ts** — reducer over money + order + journey events
- **pricing/rateCards** — priced offers, promos, tenant overrides
- order status
`PENDING→SUCCESSFUL→SETTLED` (+ counter-offer resurrection edge)

■ Cross-cutting

- **core/clock** — virtual clock drives all timers
- **core/pii · idempotency · rateLimit** (token bucket)
- **ais** open-banking sim · **metrics** · **audit · reporting · privacy** retention & subject-access

03 The three lenders & the waterfall

All three are the same policy engine parameterized differently. The router runs a leg's own lender and, *only if it declines*, probes remaining eligible lenders for a counter-offer candidate.

Ironbridge Finance

PRIME · PRIORITY 10

Score floor	≥ 700
Ticket	£250 – £5,000
Workflow	9 steps
Counter-offers	no
Settlement	weekday 17:00

Fernway Credit

NEAR-PRIME · PRIORITY 20

Score floor	≥ 420 (refer 340)
Ticket	£50 – £2,000
Latency	~3 s
Counter-offers	yes
Revolving	Fernway Flex

Brightloan

SMALL-TICKET INSTANT · PRIORITY 30

Score floor	≥ 250
Ticket	≤ £500
Workflow	3 steps
Decision delay	instant
Settlement	weekly Fri

Waterfall (`router.ts decideWithWaterfall`): the leg's own lender assesses first. On a decline, the router walks remaining lenders in priority order — filtered by ticket band, minus already-used — to the first that `acceptsCounterOffers`, recording a `counterOfferLenderId`. It freezes each `RoutedInvocation` verbatim into a `routing.trace` event and holds no outcome logic of its own. `PRODUCT_LENDERS` maps some products (e.g. revolving credit → Fernway) directly, bypassing the waterfall.

04 End-to-end application lifecycle

One purchase, journal event by journal event — from merchant auth to a serviced consumer loan.

01

Checkout

SDK /auth → /init +
/verify-basket →
merchant hands
browser a CAC (JWT)

02

Widget apply

prequal tile → plan
select → POST
product:apply HAL
link → hosted
iframe steps

03

Waterfall

build
LenderAssessInput →
assess() → probe
fallbacks →
routing.trace +
application.assessed

04

Decision

application.decided
– or manual
underwriting.decision
on a referral

05

Counter-offer

decline +
counterOfferLenderId
→ switch leg; may
resurrect
UNSUCCESSFUL→SUCCESSFUL

06

Webhook

OrderNotifier →
HMAC-signed
envelope to
callbackUri;
success carries
receipt JWT

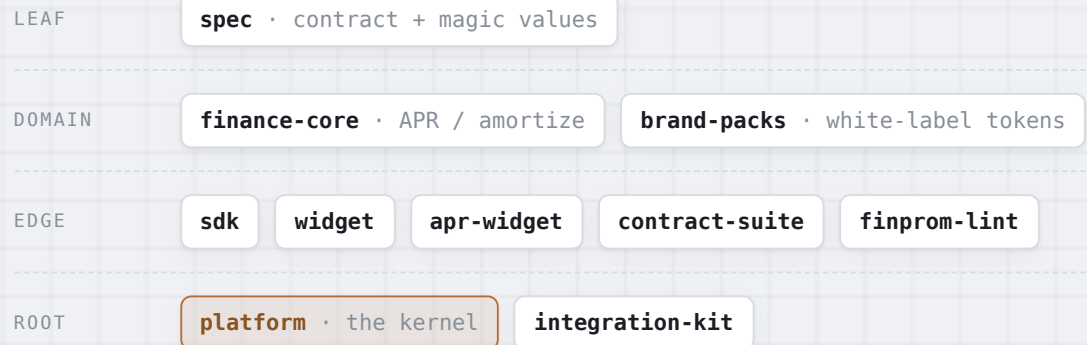
07

Settle + service

money events →
settlement.run +
reconciler;
loan.opened →
Direct Debit,
arrears, hardship

05 Package dependency graph

npm-workspaces monorepo, @lendrail/*, leaf → root. finance-core is the sole home of APR/interest math (grep-gated everywhere else).



06 Service & port map

CANONICAL PORTS — CONVENTIONS.JSON (D-105 + SUCCESSORS)

PORT	SERVICE	TYPE	TALKS TO PLATFORM VIA
7401	platform	Fastify kernel	—
7410	store — Voltora Electronics	Fastify SSR	@lendrail/sdk (server)
7411	storeApi	store back-end	@lendrail/sdk (server)
7412	dashboard	React SPA	browser HTTP
7413	lenderops	React SPA	browser HTTP
7414	store — Kestrel Cycles	Fastify SSR	@lendrail/sdk (server)
7415	demo-cart — Larkspur	Node HTTP	@lendrail/integration-kit
7420	explorer	static SPA	reads spec JSON only
7430	docs	static site	none
7431	landing	static site	none
7432	portal	static + ES modules	config.json only
7440	servicing	React PWA	browser HTTP /consumer/*

platform core / primary flow Ironbridge — prime Fernway — near-prime Brightloan — instant independently reconciled disputes / reversals

Derived from a read of `packages/*`, `apps/*` and the platform kernel — the sealed lender adapters (`src/lenders/`), settlement + servicing engines with their independent reconcilers, the webhook outbox, and the event store. Persistence is an append-only JSONL journal folded by one pure reducer; the recurring motif throughout is **a deterministic engine paired with a grep-gated, independently-derived verifier**.

Ports are authoritative from `packages/spec/conventions.json`; product/lifecycle facts from `domain/`, `lenders/`, `settlement/`, and `servicing/`.